Conception

Utiliser un abstract syntax tree

Si on utilise une analyse lexicale (regex), le problème c'est qu'elle va agir de manière trop généraliste. Par exemple si on définit une regex pour ne jamais utiliser `eval()`, il risque d'y avoir des faux-positifs sur des noms de fonction qui n'ont pas la même fonction (e.g `eval_custom()`)

L'AST résout le problème en transformant le code source en **une structure arborescente sémantique**. Avant d'effectuer l'analyse !

Donc de ce que je comprends, outil fournit un outil ast, qui permet de structure le code sous cette forme d'arbre sémantique, et **puis d'effectuer des opérations dessus**.

Design pattern visiteur

Le design permet d'éviter de devoir rajouter des if else en continu dès qu'un nouveau terme apparaît. Le module ast de Python résout ce problème de vérification lors du parcours grâce à ça. En fournissant la classe ast.NodeVisitor.

Mapper des CWE avec des CVE

L'outil est d'abord conçu comme un SAST orienté structure (ast), spécialisé dans la détection de faiblesses de code (CWE ; Common Vulnerabilities and Exposures).

La détection de CVE de bibliothèques relève des SCA. Il serait intéressant d'étendre notre SAST maison à une petite SCA. Ce qui nous amène à nous demander : Comment mapper les CWE avec des CVE ?

Solution

Concevoir les **règles de sécurité comme des objets (ou un dictionnaire de métadonnées) qui lient directement un noeud AST détecté à sa** CWE **et à une ou plusieurs** CVE historiques célèbres**

Workflow

1. **Couche d'entrée (CLI / Fichiers) :** Parcourt récursivement le système de fichiers pour récupérer l'ensemble des fichiers `.py` cibles.
2. **Couche de parsing :** Transforme le code source brut en objets `ast.AST`.

3. **Couche d'analyse (Visiteur) :** Applique les règles de détection via le pattern Visiteur pour identifier les patterns dangereux (ex: `eval()`).
4. **Couche de classification & contextualisation :**
    1. Identifie la faiblesse associée (ex : **CWE-94**).
    2. Enrichit l'alerte avec les données historiques (ex : CVE majeures liées à ce pattern).
5. **Couche de reporting :** Formate et affiche proprement les résultats (dans le terminal avec des couleurs, ou export en format JSON pour être exploitable par d'autres outils).

What is an SAST?

An SAST: **Static Application Security Testing (SAST)** is a white-box testing method that scans an application's source code, bytecode, or binaries for security vulnerabilities and coding flaws without actually running the program

Quotes:

> The obvious case for using an Abstract Syntax Tree is for creating an interpreter for a different programming language.

Links:

https://greentreesnakes.readthedocs.io/en/latest/